ProcessFlow: Problem-First Learning Plan

Goal: learn system design by *causing* real problems, measuring them, and only then adding the fix. **Duration:** ~14 working days (3–5 hrs/day). Full-time, about 8–10 days. **Side goal:** learn a professional GitHub workflow: **stacked PRs**, phase branches, **tags** and **releases**.

Working rules

1. **No code until you say so.** This plan is only the plan. Every phase starts when you say "start Phase N".
2. **No tool before a measurement proves you need it.** Redis, BullMQ, pools, caches and rate limiters only appear after a test shows the problem.
3. **Same test before and after.** Each experiment script is committed once and re-run unchanged after the fix.
4. **Record real numbers only.** Write down your machine specs and the real results, including the ones that surprise you.
5. **Timebox.** If a phase takes more than 1.5 days, cut scope and move on.

The loop you repeat every phase

```
BUILD the naive version  ->  BREAK it (load / kill / slow down)  ->  MEASURE
      ^                                                                |
RE-MEASURE with the same test  <-  APPLY one fix  <-  FIND the bottleneck
```

This loop maps directly onto the **4 stacked PRs** of each phase (see the GitHub workflow below).

For every phase there's a folder `experiments/NN-name/` with:

- `test.js` (k6) or `run.sh` (autocannon): the exact load test.
- `before.md` / `after.md`: real numbers (p50/p95 latency, req/s, errors, CPU, memory, DB connections).
- `notes.md`: what broke, why, and what the fix cost you (the trade-off).

Stack

Need	Choice	Why
API	**Express + TypeScript**	Less framework to learn; the concepts are the same in NestJS
DB	**PostgreSQL + TypeORM**	Entities + migrations; QueryBuilder for bulk inserts and `ON CONFLICT`
PDF	PDFKit	CPU-heavy, so it shows event-loop blocking clearly
Email	Nodemailer + MailHog	Local SMTP you can stop/start to simulate outages
Queue (added in Phase 4)	BullMQ + Redis	
Load test	autocannon (quick), k6 (scenarios)	
Metrics	`docker stats`, `pg_stat_activity`, `/metrics` endpoint, `perf_hooks.monitorEventLoopDelay`	Prometheus/Grafana optional at the end
Frontend	React + Vite, minimal	Built late on purpose
Code hosting	GitHub + `gh` CLI	Stacked PRs, tags, releases

Repositories

- **Backend repo** (`backend/`): API, workers, migrations, docker-compose, `experiments/`, and these docs. All 13 phases happen here. GitHub: `Moon-Khan/ProcessFlow-Distributed-Background-Job-Processing-Engine---Backend`
- **Frontend repo** (`frontend/`): React dashboard. Mostly idle until Phase 11. Same workflow (phase branch + stacked PRs), with tags prefixed `fe-` (e.g. `fe-v0.11.0`) so you can tell the two apart.

TypeORM rules for this project

- `synchronize: false` always. Every schema change is a **migration** committed in its own PR, so you can see the schema evolve phase by phase.
- Pool settings live in the DataSource: `poolSize` and `extra: { max, connectionTimeoutMillis, idleTimeoutMillis }` (passed to the `pg` driver pool). You'll tune these in Phase 2.
- Bulk writes use `createQueryBuilder().insert().into(Entity).values(batch)`. Idempotent inserts use `.orIgnore()` or a raw `INSERT ... ON CONFLICT DO NOTHING RETURNING`.

GitHub workflow: phase branches, stacked PRs and tags

The shape

```
main  <-- Phase PR (merge commit) --  p01
                                       ^  PR 1/4  p01-1-naive     (base: p01)
                                       ^  PR 2/4  p01-2-measure   (base: p01-1-naive)
                                       ^  PR 3/4  p01-3-fix       (base: p01-2-measure)
                                       ^  PR 4/4  p01-4-results   (base: p01-3-fix)
```

- `main`: only receives finished phases. Protected: no direct pushes, PR required.
- `pNN` **(phase branch)**: one per phase, cut from `main`. Its PR into `main` is the **phase PR**.
- **Stacked PRs**: each step branch is cut from the previous step's branch, and each PR's base is the branch below it. Every PR stays small and reviewable, and you can keep working on step 3 while step 1 is still in review.

> **Naming gotcha:** Git can't have a branch called `p01` *and* `p01/naive`, because `p01` would have to be both a file and a folder under `refs/heads/`. That's why step branches are `p01-1-naive`, not `p01/naive`.

The standard 4-PR stack per phase (= the learning loop)

PR	Branch	Contains	Matches loop step
1/4	`pNN-1-naive`	The naive feature (+ migration if needed)	Build
2/4	`pNN-2-measure`	Load-test script + `before.md` with real numbers	Break + Measure
3/4	`pNN-3-fix`	The single architectural fix	Apply fix
4/4	`pNN-4-results`	`after.md`, `notes.md`, README section	Re-measure + trade-off

Some phases have one extra PR (for example, two fixes measured separately). That's fine: keep one fix per PR.

Tags

Tag	Where	Meaning
`pNN-before`	on `pNN` after PR 2/4 is merged	The broken/naive state + baseline. You can always re-run the "before" test from here
`pNN-after`	on `pNN` after PR 4/4 is merged	The fixed state + results
`v0.NN.0`	on `main` after the phase PR is merged	Release. Create a **GitHub Release** whose notes are the phase's before/after table

Use **annotated** tags (`git tag -a`): they store an author, date and message, which lightweight tags don't. To reproduce any experiment later: `git switch --detach p03-before`, then run the test.

Merge strategy (and why)

- **Stacked PR -> phase branch: Squash and merge.** One clean commit per step on `pNN`.
- **Phase PR -> main: Create a merge commit.** This keeps the `pNN` commits (and the tags pointing at them) in `main`'s history. If you squashed here, `pNN-before` / `pNN-after` would point at commits that aren't in `main`.
- Repo setting: turn on **"Automatically delete head branches"**. When PR 1/4 merges and `pNN-1-naive` is deleted, GitHub automatically **retargets PR 2/4 to `pNN`**.

The two stacked-PR skills you'll practise

1. Restacking after a squash merge. After PR 1/4 is squashed into `p01`, branch `p01-2-measure` still contains the *original* (pre-squash) commits of step 1. Move only its own commits onto the new `p01`:

```
git fetch origin
git rebase --onto origin/p01 <old-tip-of-p01-1-naive> p01-2-measure
git push --force-with-lease
```

2. Fixing a lower PR in the middle of the stack. A review comment arrives on PR 1/4 while you're on step 3:

```
git switch p01-1-naive          # make the fix, commit
git switch p01-3-fix
git rebase --update-refs p01-1-naive   # restacks p01-2-measure and p01-3-fix in one go (git >=
2.38)
git push --force-with-lease origin p01-1-naive p01-2-measure p01-3-fix
```

Learn the manual way first. In the last phases you can compare it with a tool like Graphite, `spr` or `git-town`.

One phase, start to finish

> Needs the GitHub CLI: `winget install GitHub.cli` then `gh auth login`. Without it, do the `gh pr create` / `gh release create` steps in the GitHub web UI (choose the correct **base** branch when opening each PR).

```
git switch main && git pull
git switch -c p01
#   first commit on the phase branch: experiments/01-sync/README.md (hypothesis + plan)
git push -u origin p01
gh pr create --base main --head p01 --draft --title "Phase 1: Synchronous reports"

git switch -c p01-1-naive        # work, commit, push
gh pr create --base p01 --title "[P1 1/4] Naive synchronous report endpoint"
git switch -c p01-2-measure      # work, commit, push
gh pr create --base p01-1-naive --title "[P1 2/4] Load test + baseline numbers"
...
# merge bottom-up: 1/4, restack, 2/4 -> tag p01-before, 3/4, 4/4 -> tag p01-after
git tag -a p01-after -m "Phase 1 fixed + results" && git push origin p01-after
# mark phase PR ready, merge with merge commit, then on main:
git tag -a v0.1.0 -m "Phase 1" && git push origin v0.1.0
gh release create v0.1.0 --notes-file experiments/01-sync/after.md
```

(A PR needs at least one commit of difference, which is why the phase branch gets the experiment README as its first commit before the draft phase PR is opened.)

PR template (`.github/pull_request_template.md`)

```
## Problem
## What changed
## How to test / reproduce
## Measurements (before -> after)
## Trade-offs / new problems introduced
```

Phase 0: Setup, GitHub and measurement toolkit (Day 0)

Learn: repo setup, branch protection, first stack, first tags

Build

- GitHub repo, branch protection on `main`, auto-delete head branches, PR template, `.gitignore`.
- `docker-compose.yml`: postgres + mailhog (**no Redis yet**).
- Express + TypeScript app: `GET /health`, `GET /claims?page=`.
- TypeORM DataSource, entities `User`, `Claim` (id, userId, amount, status, department, createdAt), first migration.
- Seed: **100,000 synthetic claims**, size controlled by `SEED_CLAIMS` (100k / 500k / 1M). Fast way: one SQL `INSERT ... SELECT ... FROM generate_series(1, $SEED_CLAIMS)` with `random()` for user, amount, status (approved/rejected/pending), department (health/auto/home/life/travel), createdAt (last 365 days). 100k rows ≈ seconds and ~15 MB; 1M ≈ under a minute, ~150 MB.
- Request-duration middleware + `/metrics` (event-loop lag p99, memory, request counts).

PR stack (p00)

1. `p00-1-scaffold`: repo scaffold, TS config, PR template
2. `p00-2-infra`: docker-compose, DataSource, entities, migration
3. `p00-3-seed`: seed script
4. `p00-4-metrics`: metrics middleware + baseline numbers

Done when: `autocannon -c 50 -d 20 http://localhost:3000/health` baseline is in `experiments/00-baseline/before.md`, tag `v0.0.0` is on `main`.

Phase 1: Synchronous reports (Day 1–2)

Concepts: request/response coupling, event-loop blocking, timeouts, CPU-bound vs IO-bound

Naive build: `POST /reports` queries the department's claims, computes totals, generates the PDF with PDFKit, sends the email via MailHog and responds with the file path, **all inside the HTTP request**.

Why the PDF is slow (real work, not fake sleeps)

- Query ~20k claims for one department (100k ÷ 5) with no index -> full table scan.
- Compute totals per status / month / user in JS -> event-loop work.
- Render a **detail table, one line per claim** (~45 rows/page -> ~450 pages). PDFKit layout is CPU-heavy, and that's what blocks Node's single thread.
- Never use `sleep()` to make the PDF slow: a sleep uses no CPU, so it won't block the event loop and you'd miss the lesson. (The only intended artificial delay is `pg_sleep` in Phase 2, to simulate a slow DB.)

Calibration (first step of the phase): time one report. Target **~2–5 s**. Too fast -> raise `SEED_CLAIMS` or widen the report (all departments / longer date range). Too slow -> lower it. Record the chosen size in `experiments/01-sync/README.md`.

Break it

1. 1 request: note the time.
2. 10 -> 50 concurrent `/reports` requests.
3. **At the same time**, load `GET /health` from a second terminal.
4. `docker compose stop mailhog` and send a request.

Expect to discover

- `/reports` p95 grows with concurrency.
- `/health` gets slow too: PDF generation is CPU work on Node's single thread, so unrelated requests suffer (watch event-loop lag).
- With a 30s client timeout, some requests fail.
- MailHog down = the whole request fails, and the PDF work is wasted.

PR stack (p01): naive endpoint -> load test + before -> *(no fix in this phase; PR 3 is a "worker thread" spike, `piscina`, to see what it does and doesn't solve)* -> results/notes. **Lesson:** the request is tied to long work, and one slow dependency fails the whole thing.

Phase 2: The database becomes the bottleneck (Day 3)

Concepts: connection pooling, pool exhaustion, Little's Law, indexes, N+1 queries

Naive build

- `GET /claims/summary?department=` : aggregates totals, **no index** on `department` .
- First version opens a **new** `pg.Client` **per request** on purpose (bypasses the TypeORM pool).
- `GET /claims/with-users` : loads claims, then fetches each claim's user in a loop (**N+1**).

Break it

- 200 concurrent requests -> `SELECT count(*) FROM pg_stat_activity;` -> `too many clients already` (default max 100).
- Add `pg_sleep(0.2)` to simulate a slow DB and repeat.
- Turn on TypeORM query logging and count the queries for `/claims/with-users` .

Fixes (one PR each, re-measure after each)

1. Use the TypeORM pool, then try `extra.max = 2, 10, 50` . Too small: requests wait for a connection (`connectionTimeoutMillis` errors). Too big: the DB gets slower overall.
2. Index on `(department, status)` in a migration. Compare `EXPLAIN ANALYZE` before and after.
3. Replace N+1 with a join (`leftJoinAndSelect` / `relations`).

Lesson: Little's Law: connections needed ≈ request rate × query time. A pool doesn't make queries faster; it caps and reuses connections.

Phase 3: Repeated expensive reads -> Redis cache (Day 4)

Concepts: cache-aside, TTL, invalidation, cache stampede

Break it: the dashboard calls `/claims/summary` constantly. Load it at 500 req/s -> DB CPU climbs (`docker stats`), p95 rises.

Fix: add Redis to compose. Cache-aside with a 30s TTL. Re-run: DB load drops.

Then find the problems the fix created

- **Stale data:** insert claims -> summary is wrong for up to 30s. Shorter TTL or delete-on-write?
- **Stampede:** TTL 5s under heavy load -> on expiry, many requests hit the DB at once. Fix with a lock (`SET key NX PX`) or early refresh.

Phase 4: Move work off the request: queues (Day 5–6)

Concepts: async processing, 202 Accepted, producer/consumer, separate worker process, job state

Build

- BullMQ. `POST /reports` now inserts a `Job` row (TypeORM entity + migration) -> enqueues -> returns **202 + jobId** in milliseconds.
- `worker.ts` = **separate process/container**, same codebase.
- `Job` entity: `id, type, status (queued|active|completed|failed|dead), attempts, error, resultPath, createdAt, startedAt, finishedAt`.
- `GET /jobs/:id` (polling for now).
- Two queues: `pdf` and `email`. The PDF job enqueues an email job when it's done.

Re-run the Phase 1 test unchanged

- `/reports` p95: seconds -> milliseconds.
- `/health` stays fast under load (the CPU work runs in another process).
- MailHog down -> PDF still completes; only the email job fails.

New problem: the work isn't faster, it's just somewhere else. Total time = queue wait + processing time. Record both (`startedAt - createdAt` and `finishedAt - startedAt`).

Phase 5: Large CSV import (Day 7)

Concepts: streaming, memory limits, batch writes, partial-failure reporting

Naive build: `fs.readFile` the whole CSV, parse it all into memory, then `repository.save()` row by row.

Break it: a 500k-row (~100MB) CSV. Set `mem_limit: 256m` on the worker -> OOM-killed. Measure rows/sec too (row-by-row saves are slow).

Fix

- Stream-parse (`csv-parse`) -> validate each row -> insert in **batches of 1000** via `createQueryBuilder().insert().values(batch)`.
- Invalid rows -> `errors-<jobId>.csv`, downloadable.
- Progress: `job.updateProgress(rowsDone / total)`.

Measure: peak memory and rows/sec. Try batch sizes 100 / 1000 / 5000.

Phase 6: Traffic spike and worker capacity (Day 8)

Concepts: backlog, throughput, concurrency tuning, horizontal scaling, CPU-bound limits

Break it: submit **1,000 report jobs** in one burst. Start with 1 worker, `concurrency: 1`.

Measure every 5s: waiting, active, completed/sec, worker CPU and memory.

Workers	Concurrency	Jobs/sec	Max queue wait	Worker CPU	Memory
1	1				
1	4				
1	16				
3 (`--scale worker=3`)	4				

Expect to discover

- **PDF queue** (CPU-bound): more concurrency in one Node process barely helps; it's one thread. More **processes** help until your cores run out.
- **Email queue** (IO-bound): concurrency helps a lot.
- Eventually the DB becomes the bottleneck. Phase 2 comes back: check the pool and `pg_stat_activity`.

Phase 7: Transient failures: retries, backoff, DLQ (Day 9)

Concepts: retries, exponential backoff + jitter, retry storms, dead-letter queue, replay

Chaos switch: `EMAIL_FAILURE_RATE=0.5` makes the sender throw randomly. Also stop MailHog for 2 minutes.

Step by step (each its own PR + measurement)

1. **No retries** -> count lost emails.
2. **3 attempts, 0 delay** -> MailHog down with 500 queued emails -> attempts burn out in seconds = **retry storm**, and everything is lost anyway.
3. **Exponential backoff** (`attempts: 6, backoff: { type: 'exponential', delay: 1000 }`) + jitter -> a 2-minute outage -> jobs recover after MailHog returns.
4. **DLQ:** in the `failed` handler, when `attemptsMade >= attempts`, copy the job to a separate `dead-letter` queue **and** set `Job.status = 'dead'` with the error. A **non-retryable** error type (bad email address) goes straight to the DLQ.
5. `POST /admin/dlq/:id/replay` re-enqueues it with a fresh attempt count.

Evidence: lost jobs for 1–3, recovery time for 3, DLQ contents for 4.

Phase 8: Crashes and restarts (Day 10)

Concepts: stalled jobs, at-least-once delivery, durability, idempotent workers

Break it

1. Big PDF job active -> `docker kill` the worker -> restart. BullMQ detects the **stalled** job and reruns it. Measure recovery time (lock duration / stalled interval).
2. Kill the worker **after the email is sent but before the job is marked done** (add an artificial sleep there) -> **email sent twice**. Check MailHog.
3. Restart Redis without persistence -> queued jobs **vanish**, while Postgres still says `queued`.

Fix

- `EmailDelivery` entity with `@Unique(['jobId'])`. Record before send / check before send. Be honest: this **narrows** the duplicate window but can't make it zero. That's "effectively once", not "exactly once".
- Deterministic output path `reports/<jobId>.pdf`, so a rerun overwrites instead of duplicating.
- Redis `appendonly yes` + a **reconciliation script**: re-enqueue jobs that are `queued` in Postgres but missing from Redis. Lesson: Postgres is the source of truth, Redis is the transport.

Phase 9: Duplicate submissions: idempotency (Day 11)

Concepts: idempotency keys, race conditions, DB unique constraints

Break it

1. No keys: send the same request twice (double click) -> 2 jobs, 2 PDFs, 2 emails.
2. Naive fix: `findOne` the key -> if missing, `save`. Fire **20 concurrent** requests with the same key -> still duplicates. That's a **race condition**: everyone read "not found" at the same moment.

Fix

- `Idempotency-Key` header. `IdempotencyKey` entity `(userId, key, requestHash, jobId)` with `@Unique(['userId', 'key'])`.
- `INSERT ... ON CONFLICT DO NOTHING RETURNING` (TypeORM `.orIgnore()` + reselect, or raw query) in a transaction -> one winner creates the job; everyone else gets the **same jobId**.
- Same key + different body (`requestHash` mismatch) -> `422`.
- Also pass `jobId` to BullMQ so the queue rejects a duplicate enqueue.

Evidence: requests sent vs jobs created vs PDFs on disk vs emails in MailHog.

Phase 10: One user floods the system: rate limiting (Day 12)

Concepts: token bucket / sliding window, fairness, 429 + Retry-After, protecting downstream services

Break it: user A submits 5,000 jobs; user B submits 1. Measure **B's wait time**: it's stuck behind A's backlog.

Fixes (one PR each, measure after each)

1. Per-user API rate limit in Redis (build it yourself: `INCR` + `EXPIRE` or a Lua token bucket) -> `429` + `Retry-After`. Then compare with `rate-limiter-flexible`.
2. Per-user cap on in-flight jobs (e.g. 50 queued) -> backpressure.
3. Downstream limiter: BullMQ `limiter: { max: 10, duration: 1000 }` on the email queue (a "provider limit").

Re-measure: B's wait time and total throughput. Did fairness cost throughput?

Phase 11: Live status: polling vs push (Day 13)

Concepts: polling cost, WebSockets, pub/sub, fan-out across API instances

Break it: 300 simulated dashboard tabs poll `GET /jobs/:id` every second (k6). Measure API req/s and DB queries/s spent only on "is it done yet?".

Fix

- Socket.IO on the API, fed by BullMQ `QueueEvents` (`active`, `progress`, `completed`, `failed`) -> push to the job owner's room.
- Minimal React dashboard: job list, status, progress, attempts, error, download link, DLQ list with Replay.
- Run **2 API instances** -> clients on instance 2 miss events -> add the Socket.IO Redis adapter.

Evidence: req/s and DB load, polling vs WebSockets.

Phase 12: Observability and write-up (Day 14)

- Optional Prometheus + Grafana: queue depth, job duration, failures, API p95.
- README: the business problem (SecureLife Insurance), architecture diagram, one section per phase (problem -> measurement -> fix -> re-measurement -> trade-off), links to each `vX.Y.0` release.
- Only publish numbers you measured, with machine specs.
- Final tag `v1.0.0`.

Schema timeline (tables appear when a problem needs them)

The schema grows **incrementally through migrations**. It is not designed upfront. Each migration ships inside the PR of the phase that needed it.

Phase	Migration	Why then
0	`users`, `claims`	The business data reports are built from
1	`reports` (id, user_id, department, file_path, created_at)	Record generated PDFs
2	Index on `claims (department, status)`	`EXPLAIN ANALYZE` shows a full scan
4	`jobs` (type, status, attempts, error, result_path, created_at, started_at, finished_at)	Work moved to a queue and needs a lasting status record
5	`import_batches` (+ error_report_path, rows ok/failed)	CSV imports need progress + an error report
7	`jobs.status` gains `dead`, plus `last_error_at`	DLQ + replay
8	`email_deliveries` (job_id **UNIQUE**, sent_at)	A crash caused a duplicate email
9	`idempotency_keys` (user_id, key, request_hash, job_id, **UNIQUE(user_id, key)**)	Double click / concurrent requests created 2 jobs
10	Maybe `users.max_inflight_jobs` / `users.plan`	Per-user limits

Exact columns are decided in each phase, based on what the experiment shows.

Migration lessons along the way

- Changing an existing column type on a table with data (Phase 7). If `jobs.status` is a Postgres enum, adding a value needs care, which is why many teams use `varchar` + `CHECK` instead.
- `CREATE INDEX` on a big table blocks writes while it builds. `CREATE INDEX CONCURRENTLY` doesn't, but it can't run inside a transaction, and TypeORM migrations run in a transaction by default, so turn that off for that one migration (Phase 2).
- **Never edit a merged migration.** Always add a new one. Generate with `typeorm migration:generate`.

Cheat sheet: problem -> concept -> fix

You observe...	Bottleneck	Fix	New trade-off
Slow API, unrelated endpoints slow too	CPU work on the event loop	Queue + separate worker	Job state, eventual completion
`too many clients` / connection waits	DB connections	Pool + index + no N+1	Pool sizing
DB CPU high from repeated reads	Repeated computation	Redis cache	Stale data, stampede
Worker OOM on a big file	Memory	Streaming + batching	More complex code
Queue keeps growing	Worker capacity	Concurrency / more workers	DB becomes the next bottleneck
Lost work during a temporary outage	Transient failure	Retry + backoff + DLQ	Delays, duplicate side effects
Duplicate emails after a crash	At-least-once delivery	Idempotent worker steps	Extra state
Double click -> 2 jobs	Race condition	Idempotency key + unique constraint	Key design per operation
One user starves others	Fairness	Rate limit + per-user caps	Users see 429s
Polling overload	Wasted requests	WebSockets + pub/sub	Stateful connections

Phase / tag / release map

Phase	Branch	Tags	Release
0 Setup	`p00`	`p00-after`	`v0.0.0`
1 Sync reports	`p01`	`p01-before`, `p01-after`	`v0.1.0`
2 DB bottleneck	`p02`	`p02-before`, `p02-after`	`v0.2.0`
3 Cache	`p03`	`p03-before`, `p03-after`	`v0.3.0`
4 Queues	`p04`	`p04-before`, `p04-after`	`v0.4.0`
5 CSV import	`p05`	`p05-before`, `p05-after`	`v0.5.0`
6 Spike + scaling	`p06`	`p06-before`, `p06-after`	`v0.6.0`
7 Retries + DLQ	`p07`	`p07-before`, `p07-after`	`v0.7.0`
8 Crashes	`p08`	`p08-before`, `p08-after`	`v0.8.0`
9 Idempotency	`p09`	`p09-before`, `p09-after`	`v0.9.0`
10 Rate limiting	`p10`	`p10-before`, `p10-after`	`v0.10.0`
11 Live status	`p11`	`p11-before`, `p11-after`	`v0.11.0`
12 Write-up	`p12`	-	`v1.0.0`